\# Reporte y Gestión de Defectos

\## VotaGT - Sistema de Gestión Electoral

Estándar de referencia: IEEE 1044-2009

Curso: Análisis de Sistemas II

Versión: 1.0

Fecha: Octubre 2026

\---

\# 1. Propósito

El presente documento define el proceso utilizado en VotaGT para identificar, clasificar, priorizar, corregir y cerrar defectos de software.

La gestión de defectos permite:

\- Registrar errores encontrados.

\- Determinar su impacto.

\- Asignar prioridad.

\- Dar seguimiento a la corrección.

\- Evitar regresiones.

\- Mantener evidencia de calidad.

\---

\# 2. Definición de defecto

Un defecto es una condición del software que provoca que el comportamiento real sea diferente del comportamiento esperado.

Ejemplos:

\- Código HTTP incorrecto.

\- Validación inexistente.

\- Error en persistencia.

\- Falla de autenticación.

\- Respuesta incorrecta.

\- Caída del sistema.

\- Error de integración.

\- Comportamiento no documentado.

\---

\# 3. Origen de los defectos

Los defectos pueden detectarse mediante:

\- Pruebas unitarias.

\- Pruebas de integración.

\- Postman.

\- Escenarios BDD.

\- Pruebas negativas.

\- Swagger.

\- Pruebas de resiliencia.

\- Revisión manual del código.

\- Pruebas de bases de datos.

\---

\# 4. Clasificación de defectos

\## 4.1 Funcional

Ocurre cuando una funcionalidad no cumple el comportamiento esperado.

Ejemplo:

`Registrar resultado devuelve un código incorrecto.`

\---

\## 4.2 Validación

Ocurre cuando el sistema acepta datos que deberían rechazarse.

Ejemplo:

`Permitir votos negativos.`

\---

\## 4.3 Seguridad

Relacionado con autenticación, autorización o exposición de información.

Ejemplo:

`Permitir acceder a un endpoint protegido sin JWT.`

\---

\## 4.4 Persistencia

Relacionado con PostgreSQL, MongoDB, Redis o Neo4j.

Ejemplo:

`Un resultado no queda guardado en PostgreSQL.`

\---

\## 4.5 Integración

Ocurre cuando dos componentes no se comunican correctamente.

Ejemplo:

`ResultadoService no invalida correctamente Redis.`

\---

\## 4.6 Rendimiento

Relacionado con tiempos de respuesta o uso ineficiente de recursos.

Ejemplo:

`Las consultas repiten acceso a PostgreSQL aunque exista caché.`

\---

\## 4.7 Resiliencia

Relacionado con el comportamiento del sistema cuando una dependencia falla.

Ejemplo:

`La API deja de responder cuando Redis está fuera de servicio.`

\---

\## 4.8 Documentación

Ocurre cuando la documentación no coincide con la implementación.

Ejemplo:

`OpenAPI contiene un endpoint que no existe en el backend.`

\---

\# 5. Severidad

La severidad representa el impacto técnico del defecto.

\## Crítica

El sistema no puede operar o existe riesgo grave para la información.

Ejemplos:

\- API no inicia.

\- Pérdida de resultados.

\- Autenticación completamente inoperable.

\- Acceso no autorizado a información crítica.

\---

\## Alta

Una funcionalidad importante no funciona y no existe una alternativa adecuada.

Ejemplos:

\- No se pueden registrar resultados.

\- No se pueden consultar votaciones.

\- Error de autorización.

\---

\## Media

La función presenta problemas, pero existe una alternativa o el sistema puede continuar.

Ejemplos:

\- Redis no funciona pero PostgreSQL sí.

\- Exportación falla mientras la consulta normal funciona.

\---

\## Baja

Problemas menores que no impiden la operación.

Ejemplos:

\- Texto incorrecto.

\- Mensaje poco claro.

\- Error visual.

\- Formato no uniforme.

\---

\# 6. Prioridad

La prioridad representa qué tan rápido debe corregirse un defecto.

\## P1 - Inmediata

Debe corregirse antes de continuar con la entrega.

Normalmente asociada a severidad crítica.

\## P2 - Alta

Debe corregirse dentro del incremento actual.

\## P3 - Media

Puede corregirse después de las funcionalidades críticas.

\## P4 - Baja

Puede planificarse para una mejora posterior.

\---

\# 7. Matriz Severidad / Prioridad

Severidad	Prioridad sugerida
Crítica	P1
Alta	P1 / P2
Media	P2 / P3
Baja	P3 / P4

La severidad y la prioridad no son exactamente lo mismo.

La severidad mide el impacto técnico.

La prioridad determina cuándo debe atenderse.

\---

\# 8. Ciclo de vida de un defecto

El ciclo de vida utilizado es:

```text
Nuevo
&#x20; ↓
Analizado
&#x20; ↓
Asignado
&#x20; ↓
En corrección
&#x20; ↓
Corregido
&#x20; ↓
En verificación
&#x20; ↓
Cerrado
```

También puede existir:

```text
Reabierto
```

cuando la validación demuestra que el problema continúa.

\---

\# 9. Estados

\## Nuevo

El defecto fue identificado y registrado.

\## Analizado

Se confirmó que el comportamiento representa un problema real.

\## Asignado

Se asignó la responsabilidad de corregirlo.

\## En corrección

Se está modificando el código.

\## Corregido

Se realizó el cambio correspondiente.

\## En verificación

QA ejecuta nuevamente la prueba.

\## Cerrado

La corrección fue validada satisfactoriamente.

\## Reabierto

El defecto reapareció o la corrección fue insuficiente.

\---

\# 10. Plantilla de registro

Cada defecto deberá registrar como mínimo:

Campo	Descripción
ID	Identificador único
Título	Descripción corta
Fecha	Fecha de detección
Módulo	Componente afectado
Descripción	Detalle del problema
Pasos	Cómo reproducirlo
Resultado esperado	Comportamiento correcto
Resultado obtenido	Comportamiento observado
Severidad	Crítica / Alta / Media / Baja
Prioridad	P1 / P2 / P3 / P4
Estado	Ciclo de vida
Evidencia	Captura, log o prueba
Corrección	Solución aplicada

\---

\# 11. Defectos encontrados durante VotaGT

\## DEF-001 - Importación incorrecta del middleware de permisos

Módulo: Pruebas unitarias / Permission Middleware

Clasificación: Integración

Severidad: Media

Prioridad: P2

Estado: Cerrado

\### Descripción

Las pruebas unitarias del middleware fallaban porque el módulo exportaba:

`{ requierePermiso }`

pero la prueba lo importaba como exportación directa.

\### Resultado esperado

La función `requierePermiso` debía poder ejecutarse en las pruebas.

\### Resultado obtenido

Las pruebas fallaban.

\### Corrección

Se cambió la importación a:

```javascript
const {
&#x20;   requierePermiso
} = require(
&#x20;   '../../src/middlewares/permission.middleware'
);
```

\### Verificación

`npm test` finalizó sin fallos.

\---

\## DEF-002 - Archivo de configuración detectado como prueba

Módulo: Testing
Clasificación: Configuración
Severidad: Baja
Prioridad: P3
Estado: Cerrado

\### Descripción

El archivo:

`src/config/test-db.js`

era detectado automáticamente por el ejecutor de pruebas de Node.

\### Resultado esperado

Solo los archivos de pruebas reales debían contarse.

\### Corrección

Se renombró como:

`src/config/verificar-db.js`

\### Verificación

La ejecución de `npm test` mostró únicamente las pruebas correspondientes.

\---

\## DEF-003 - Nodemon reiniciaba al modificar archivos de documentación

Módulo: Desarrollo

Clasificación: Configuración

Severidad: Media

Prioridad: P2

Estado: Cerrado

\### Descripción

Nodemon observaba archivos fuera de `src`, provocando reinicios cuando se exportaba la colección Postman en OneDrive.

\### Corrección

Se modificó el script:

```json
"dev": "nodemon --watch src --ext js src/app.js"
```

\### Verificación

Los cambios en `docs/` dejaron de reiniciar la API.

\---

\## DEF-004 - Redis impedía iniciar correctamente en modo degradado

Módulo: Redis / Resiliencia

Clasificación: Resiliencia

Severidad: Alta

Prioridad: P1

Estado: Cerrado

\### Descripción

Cuando Redis estaba detenido, el cliente intentaba reconectarse repetidamente y podía interferir con el inicio normal de la API.

\### Resultado esperado

Redis debía ser auxiliar y la API debía iniciar aun cuando estuviera fuera de servicio.

\### Corrección

Se modificó la estrategia de conexión para permitir:

`Redis: DEGRADED`

y continuar utilizando PostgreSQL.

\### Verificación

Se detuvo Redis y la API inició correctamente.

\---

\## DEF-005 - Circuit Breaker no detectaba inmediatamente Redis inactivo

Módulo: Cache Gateway

Clasificación: Resiliencia

Severidad: Media

Prioridad: P2

Estado: Cerrado

\### Descripción

Las operaciones podían depender del comportamiento interno de reconexión del cliente Redis.

\### Corrección

Se incorporó validación de disponibilidad mediante:

`redisClient.isReady`

antes de ejecutar operaciones.

\### Verificación

Se observaron:

```text
FALLO 1/3
FALLO 2/3
FALLO 3/3
CIRCUIT OPEN
```

mientras la API continuó respondiendo mediante PostgreSQL.

\---

\## DEF-006 - Uso directo de AuditService desde los servicios

Módulo: Auditoría

Clasificación: Diseño

Severidad: Media

Prioridad: P2

Estado: Cerrado

\### Descripción

Los servicios llamaban directamente a `audit.service.js`, generando acoplamiento con la persistencia de auditoría.

\### Corrección

Se implementó:

```text
EventBus
&amp;#x20;  ↓
Audit Observer
&amp;#x20;  ↓
Audit Service
```

Los servicios ahora publican eventos `AUDITORIA`.

\### Verificación

Los eventos se registraron correctamente en MongoDB.

\---

\# 12. Validación de correcciones

Después de corregir un defecto se deberán ejecutar las pruebas relacionadas.

Dependiendo del cambio se utilizarán:

\- `npm test`.

\- Postman.

\- Swagger.

\- PostgreSQL.

\- Redis.

\- MongoDB.

\- Neo4j Browser.

Una corrección no se considerará cerrada hasta que la prueba correspondiente finalice satisfactoriamente.

\---

\# 13. Pruebas de regresión

Después de modificaciones importantes se deberán ejecutar nuevamente las pruebas existentes.

Ejemplo:

```cmd
npm test
```

También deberán validarse los endpoints principales mediante Postman cuando el cambio afecte la API.

\---

\# 14. Métricas

Se podrán registrar:

\- Total de defectos encontrados.

\- Defectos abiertos.

\- Defectos cerrados.

\- Defectos por severidad.

\- Defectos reabiertos.

\- Defectos encontrados por módulo.

Para la entrega final:

```text
Defectos críticos abiertos = 0
```

\---

\# 15. Relación con SQA

La gestión de defectos forma parte del proceso de Quality Control.

El Plan SQA define los mecanismos preventivos y de validación.

El reporte de defectos registra los problemas encontrados durante dichas actividades.

La relación general es:

```text
```

QA

 ↓

Prevención

Testing / QC

 ↓

Detección

Defect Management

 ↓

Corrección y seguimiento

```

\---

\# 16. Conclusión

La gestión de defectos de VotaGT permite documentar problemas reales detectados durante el desarrollo y demostrar su corrección mediante evidencia.

La clasificación, severidad, prioridad y ciclo de vida permiten administrar los defectos de manera estructurada y mantener la calidad del sistema durante su evolución.
```